Snake Game Using Hamiltonian Cycles using RL

Reinforcement Learning

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY,
DESIGN AND MANUFACTURING,
KANCHEEPURAM

CS22B1021 Nivedh Biju
CS22B1024 Nuaim

IIITD&M Kancheepuram
INDIAN INSTITUTE OF INFORMATION TECHNOLOGY,
DESIGN AND MANUFACTURING,
KANCHEEPURAM

PROBLEM STATEMENT

This project aims to solve the Snake game by training a reinforcement learning agent to follow Hamiltonian cycles on a grid. Using a CNN-based Deep Q-Network, the agent learns to cover all grid cells without collision, ensuring complete coverage and survival. The approach combines deep reinforcement learning with combinatorial pathfinding to create efficient and generalizable snake movement strategies.

INTRODUCTION

Traditional RL in Snake:

- Most reinforcement learning (RL) approaches to the Snake game use a reward structure based on collecting "apples" (food items).
- The agent is rewarded for growing longer and penalized for collisions or going out of bounds.
- This often leads to short-sighted policies that focus on immediate reward collection rather than long-term survival or optimal coverage.

Limitations of Apple-Based Rewards:

- Agents trained this way may fail to generalize to longer games or larger grids.
- They struggle with cornering, trapping themselves, or failing to utilize the full space efficiently.
- The game becomes increasingly complex as the snake grows, and reactive strategies begin to fail.

OBJECTIVE

Hamiltonian Cycles as a Solution:

- A Hamiltonian cycle visits every cell on the grid exactly once before returning to the start—ideal for the Snake game where the goal is to survive as long as possible.
- Following a Hamiltonian cycle guarantees that the snake never collides with itself and fills the board completely.
-

Reinforcement Learning + Hamiltonian Cycles:

- Instead of hand-coding such paths, we use reinforcement learning to train an agent to learn Hamiltonian-like traversal strategies.
- The agent is trained on small grids and progressively transferred to larger ones.
- A tailored reward structure encourages behaviors that contribute to cycle formation and complete coverage.

WHY IS IT IMPORTANT?

This approach generalizes to various path-planning and coverage problems, such as:

- Robotic vacuum cleaning or lawn mowing (complete area coverage without overlap)
- Drone surveillance over a grid-based map
- PCB routing or network packet traversal
- Optimization of delivery or inspection routes in logistics
- It shows how framing problems in terms of structured paths (like Hamiltonian cycles) can lead to more robust and scalable RL solutions.

ENVIRONMENT

Grid World Setup

- The game is played on an N × N square grid (e.g., 2×2, 4×4, 6×6, ..., up to 10×10 or higher).
- Each cell on the grid can be in one of the following states:
- Empty
- Occupied by the snake's body
- Apple (optional, if using apple-based rewards)

Snake Properties

- The snake is represented as an ordered list of grid positions, starting with the head and followed by body segments.
- The snake starts with a fixed initial length (usually 1 or 2).
- Movement is discrete in four directions: up, down, left, right.
- The snake's body moves forward in the chosen direction, and if it eats an apple, it grows.

ENVIRONMENT

Actions

- The agent’s action space consists of 4 discrete actions:
- makefile
- CopyEdit
- 0: Up1: Down2: Left3: Right
- Illegal actions (e.g., reversing direction into its own body) can either be:
  - Filtered out, or
  - Penalized during training.

Observations (State Representation)

- The agent observes the current game state as a 2D matrix or image input.
- State can be encoded in various formats:
  - Binary Grid:
    - 0: empty cell
    - 1: snake body
    - 2: snake head
    - 3: apple (if used)
  - Multi-Channel Tensor (better for CNNs):
    - Channel 1: snake body
    - Channel 2: apple
    - Channel 3: directional information (e.g., heading vector)
- This matrix is passed into a CNN (Convolutional Neural Network) within the RL model.

METHODOLOGY 1

Reward Structure

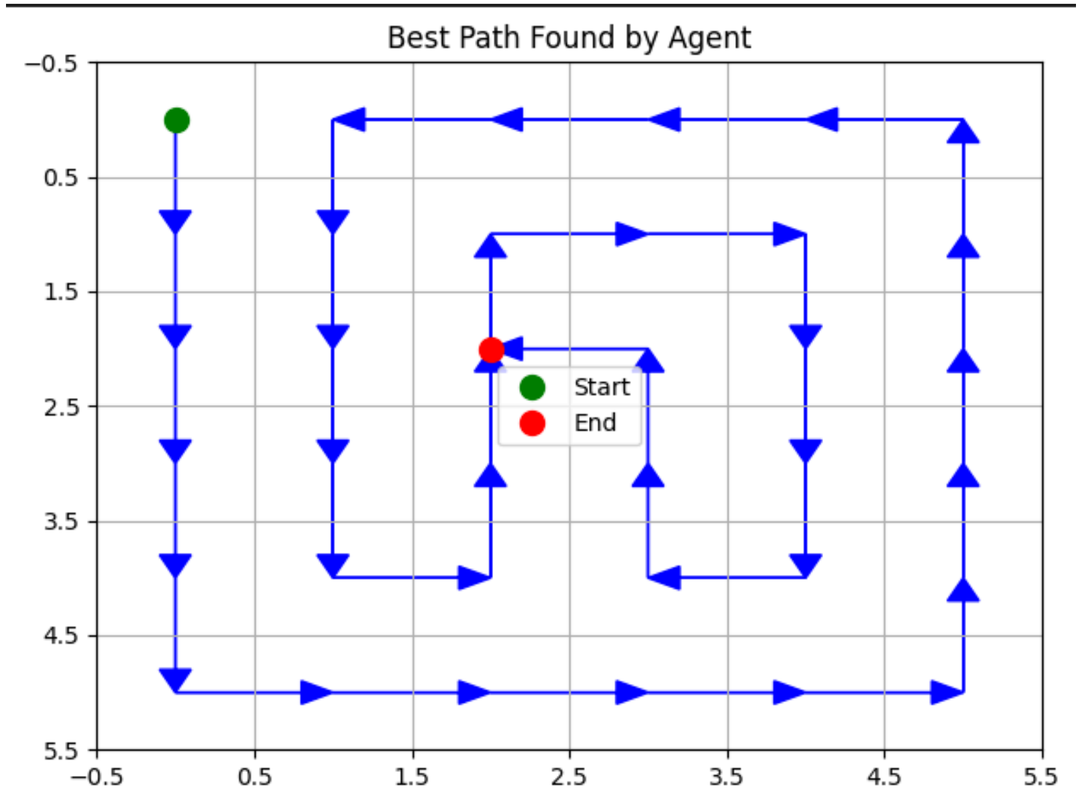
- Moving to an unvisited cell : +1
- Moving to an already visited cell : -1
- Completing a Hamiltonian cycle (all 36 cells visited and return to start) : +10
- Collision with wall/body : Episode ends (no additional reward)

. **Learning Algorithm**

- Use Deep Q-Network (DQN) with a artificial neural network architecture.
- Replay buffer and epsilon-greedy exploration strategy are used.
- The target network is updated periodically to stabilize learning.

RESULTS

- The agent consistently learned to cover all 36 grid cells without collisions.
- In a majority of evaluation episodes, the snake was able to visit each cell exactly once, demonstrating strong path planning and exploration capabilities.
- However, the agent did not reliably complete a full Hamiltonian cycle — i.e., it failed to return to the starting cell after covering all squares.



INTEPRETATION

- The problem inherently involves two simultaneous objectives:

1. Filling all grid cells exactly once (complete coverage).
2. Returning to the starting cell to form a closed Hamiltonian cycle.

- While the agent successfully learns to explore and fill the entire grid, it struggles with the cycle completion aspect. This is likely because:
- The agent uses most of its learning capacity and exploration effort on reaching all unvisited cells.
- By the time it reaches the last few cells, no planned trajectory remains to return to the start without revisiting a cell or crashing.
- This suggests that the model lacks the ability to plan global paths that balance both goals from the beginning of the episode.

Proposed Solution:

- Train the agent on smaller grids first (e.g., 2×2, 4×4), where learning both coverage and cycle completion is easier and less computationally demanding.
- Then use transfer learning or curriculum learning to scale the policy up to larger grids like 6×6, progressively learning more complex spatial strategies.
- This gradual progression helps the agent internalize both objectives as it grows in capability.

METHODOLOGY 2

This approach refines the Hamiltonian cycle objective by leveraging curriculum learning, deep Q-learning, and dense reward shaping. The core idea is to progressively train a convolutional DQN agent on smaller grids and transfer its knowledge to larger ones.

Reward Function:

- +100 for visiting a new cell.
- -1000 for crashing (invalid move or revisiting).
- +1000 × (visited_ratio) for reaching back to start without full coverage.
- +10000 for completing a full Hamiltonian cycle (visiting all cells and returning to start).
- This reward design provides dense feedback during training, encouraging efficient coverage and eventually closing the loop

METHODOLOGY 2

Agent Architecture:

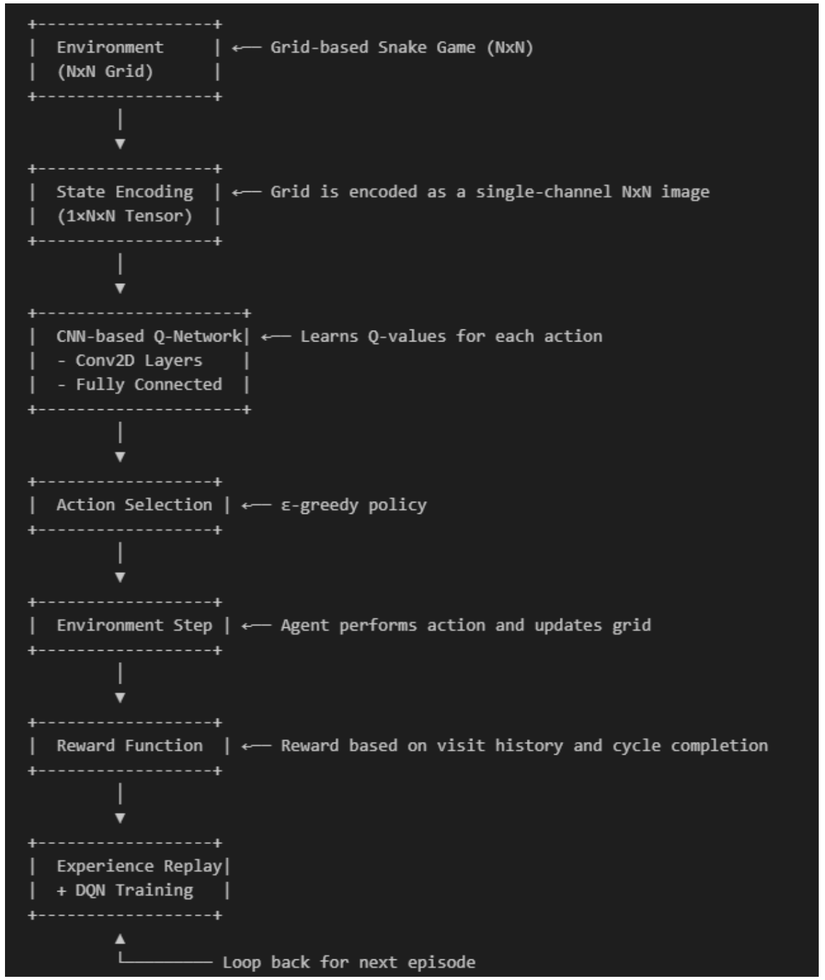
- A lightweight CNN-based DQN takes the grid as input and outputs Q-values for the 4 possible actions.
- The architecture includes:
  - Two convolutional layers
  - Two fully connected layers
  - ReLU activations
- It is trained using experience replay and epsilon-greedy exploration.

Transfer Learning Strategy:

- A base model is pretrained on smaller grids (like 2×2) and its weights are partially transferred to initialize training on larger grids (e.g., 4×4, 6×6).
- This approach reduces the learning burden on the agent and encourages faster convergence on complex grids.

ARCHITECTURE

Layer	Configuration	Output Shape
Input	Grid reshaped to `(1 × GRID_SIZE × GRID_SIZE)`	`1 × GRID_SIZE × GRID_SIZE`
Conv2D #1	`32 filters`, `3×3 kernel`, `padding=1`	`32 × GRID_SIZE × GRID_SIZE`
ReLU	Activation	
Conv2D #2	`64 filters`, `3×3 kernel`, `padding=1`	`64 × GRID_SIZE × GRID_SIZE`
ReLU	Activation	
Flatten	`64 * GRID_SIZE * GRID_SIZE`	`1D vector`
FC #1	Fully Connected Layer, `256 units`	`256`
ReLU	Activation	
FC #2 (Output)	Fully Connected Layer, `4 units` (actions)	`4` (Q-values for Up, Down, Left, Right)



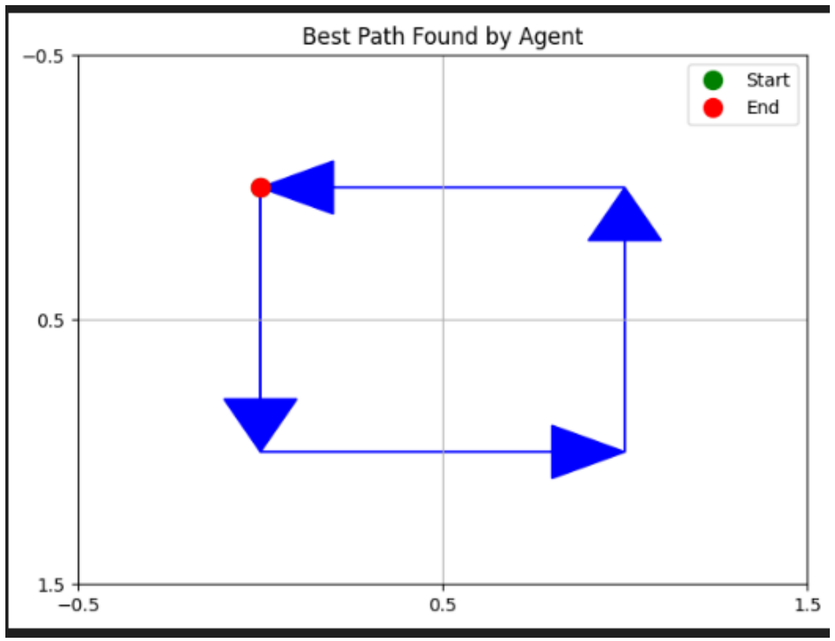
WHY CNN ?

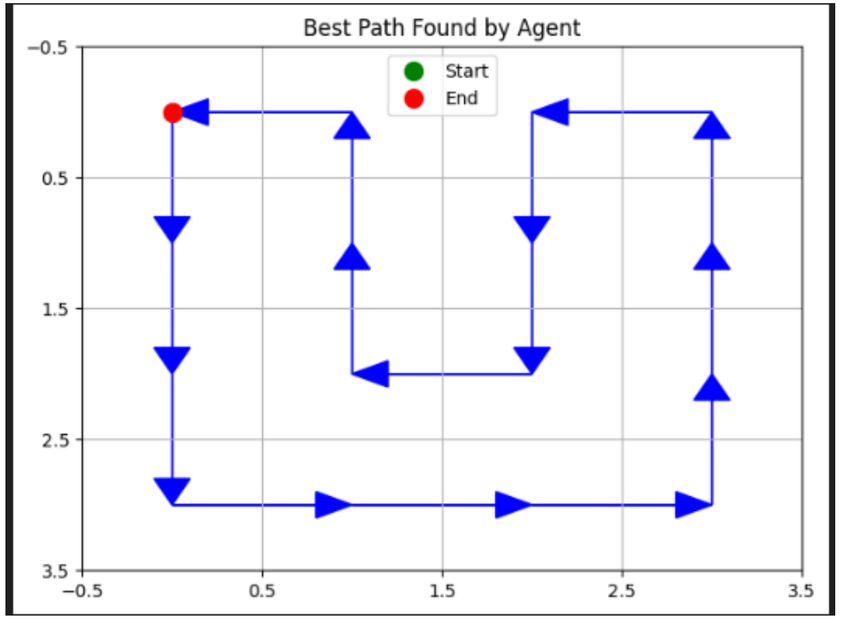
- **Spatial Invariance:** CNNs capture local patterns (e.g., walls, unvisited cells), essential for navigation in a 2D grid.
- **Scalability:** CNNs can handle variable input sizes (2×2 to 6×6 grids) without needing a fixed-size input, enabling transfer learning across grid sizes.
- **Efficient Parameters:** CNNs use fewer parameters due to weight sharing, making them more efficient compared to fully connected networks.
- **Topological Awareness:** CNNs preserve the spatial structure of the grid, helping the agent learn local movements (e.g., avoid revisiting cells) and eventually form Hamiltonian cycles.

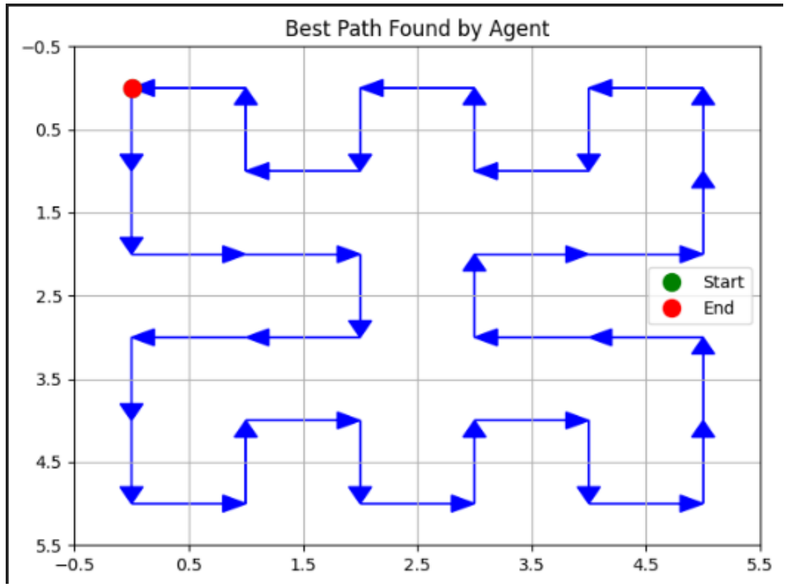
RESULTS

- The model successfully covered all cells on the grid, exploring every possible position.
- After training and refinement, the model was able to form a Hamiltonian cycle on the 6×6 grid.
- The snake traversed the grid in such a way that it visited each cell exactly once and returned to the starting point, completing the cycle.
- The approach demonstrated the feasibility of using reinforcement learning to solve complex grid-based pathfinding problems like Hamiltonian cycles.
- This success was achieved through progressive training on smaller grids (e.g., 2×2), with knowledge transferred to larger grids.

RESULTS

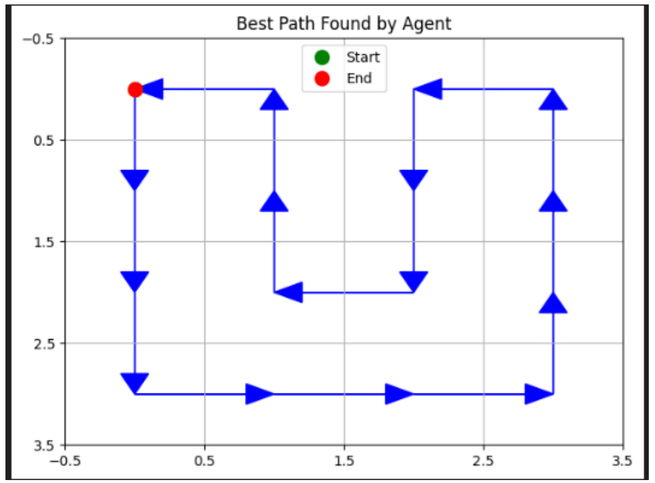

Best Path Found by Agent
Start
End
−0.5
0.5
1.5
−0.5
0.5
1.5

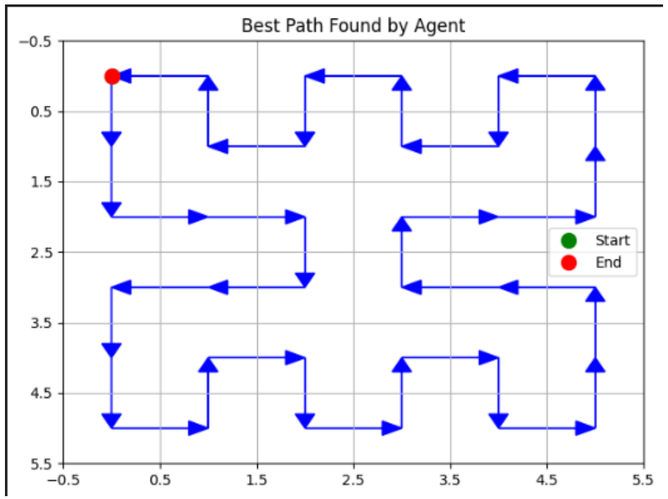

Best Path Found by Agent
Start
End
−0.5
0.5
1.5
2.5
3.5
−0.5
0.5
1.5
2.5
3.5


Best Path Found by Agent
Start
End
−0.5
0.5
1.5
2.5
3.5
4.5
5.5
−0.5
0.5
1.5
2.5
3.5
4.5
5.5

OBSERVATIONS

- A clear pattern from the 4×4 grid was observed in the 6×6 grid, indicating successful knowledge transfer from smaller to larger grids.
- The model first utilized the learned Hamiltonian cycle pattern from the 4×4 grid on the 6×6 grid, efficiently covering the entire grid while maintaining the cycle structure.
- As the agent traversed the larger grid, it focused on filling the remaining unvisited cells after completing the cycle, ensuring the Hamiltonian cycle was preserved throughout the process.
- This approach demonstrates that the model learns the fundamental structure of the cycle on smaller grids and scales it up effectively to larger grids.





REFERENCES

- An Introduction to Q-Learning (https://www.freecodecamp.org/news/an-introduction-to-q-learning-reinforcement-learning-14ac0b4493cc/)
- Proximal Policy Optimization in RL (https://www.geeksforgeeks.org/a-brief-introduction-to-proximal-policy-optimization/)
- Deep Q-Learning (https://www.geeksforgeeks.org/deep-q-learning/)
- M. Rahman, M. Kaykobad, On Hamiltonian cycles and Hamiltonian paths, Information Processing Letters 94 (2005)
- Ruikang Zhang, Ruikai Cai, Train a snake with reinforcement learning algorithms, Department of Information Engineering The Chinese University of Hong Kong Shatin